Name:J.Vishnu Vardhan
Reg No:192324297

Annexure A

Constraint-Based Problem Solving

1. Secure Face Recognition in a Low-Light Environment

Scenario

A company wants to deploy a secure access system using face recognition in a low-light environment.

Constraints:

- Limited computational resources on an embedded device
- Face orientation may vary by ±30°
- Recognition time must be less than 200 ms per face

1.1 Adapting Eigenface-Based Recognition for Illumination and Orientation

Eigenface recognition is based on **Principal Component Analysis (PCA)**, where facial images are represented using important principal components. To handle different illumination conditions, the images can first be normalized using **histogram equalization or illumination normalization**. This reduces the effect of brightness differences.

For orientation variations of ±30°, the system can:

1. Detect facial landmarks such as the eyes, nose, and mouth.
2. Align the detected face to a standard frontal position.
3. Rotate and normalize the face before recognition.
4. Train the Eigenface model using images captured at different angles.

A multi-view Eigenface model can also be used, where separate models are trained for frontal, left-facing, and right-facing faces.

Conclusion: Face alignment, illumination normalization, and multi-view training can improve Eigenface recognition under different lighting and orientation conditions.

1.2 Preprocessing Steps for Low-Light Conditions

The following preprocessing steps can improve recognition performance:

1. **Face Detection:** Detect the face and remove unnecessary background.
2. **Grayscale Conversion:** Convert the image to grayscale to reduce computational complexity.
3. **Image Enhancement:** Use histogram equalization or CLAHE to improve contrast.
4. **Denoising:** Apply Gaussian or median filtering to remove sensor noise.
5. **Gamma Correction:** Increase brightness in dark regions.
6. **Face Alignment:** Align the eyes and facial landmarks.
7. **Image Resizing:** Resize the face to a fixed resolution required by the Eigenface model.
8. **Normalization:** Normalize pixel intensity values.

A possible pipeline is:

Input Image → Denoising → CLAHE/Gamma Correction → Face Detection → Alignment → Normalization → Eigenface Recognition

---

1.3 Optimization for Real-Time Performance

To achieve recognition in less than 200 ms on constrained hardware:

- Use a low-resolution grayscale image.
- Use PCA to reduce the number of facial features.
- Store only the most important Eigenfaces.
- Use efficient face detection and landmark detection methods.
- Apply integer or fixed-point arithmetic where possible.
- Quantize the model to reduce memory usage.
- Use hardware acceleration such as GPU, NPU, or DSP if available.
- Perform face detection only when motion is detected.
- Cache previously recognized users.

Result: A lightweight Eigenface model combined with image preprocessing and PCA feature reduction can provide fast recognition on an embedded device.

---

2. Photo Album Organization

Scenario

An AI system must automatically organize a large personal photo collection by people and events.

Constraints:

- Millions of images with different resolutions and quality
- Multiple images of the same person in different poses
- All processing must be performed locally for privacy

2.1 Face Recognition and Clustering

The system can use the following process:

Step 1: Face Detection

A face detector identifies all faces in every image.

Step 2: Face Alignment

Facial landmarks are used to align faces to a common orientation.

Step 3: Feature Extraction

A face recognition model converts each face into a numerical feature vector called an **embedding**.

Step 4: Similarity Comparison

The similarity between embeddings is calculated using:

- Cosine similarity, or
- Euclidean distance

Step 5: Clustering

Similar face embeddings are grouped using clustering algorithms such as:

- DBSCAN
- Agglomerative clustering
- Hierarchical clustering

Photos containing faces from the same cluster are then grouped as the same individual.

Pipeline:

Photos → Face Detection → Alignment → Feature Extraction → Similarity Measurement → Clustering → Person-Based Albums

This method can handle different poses because embeddings represent important facial characteristics rather than exact pixel values.

2.2 Detecting Occlusions or Partial Faces

The system should first estimate the quality and completeness of each detected face.

Possible methods include:

1. **Face Landmark Detection:** Missing landmarks may indicate occlusion.
2. **Face Quality Score:** Estimate blur, brightness, resolution, and visible facial area.
3. **Occlusion Detection:** Detect sunglasses, masks, hands, or other objects covering the face.
4. **Pose Estimation:** Measure whether the face is partially turned away.
5. **Classification:** Categorize faces as:
    - Clear face
    - Partially occluded
    - Heavily occluded
    - Low-quality face

Images containing heavily occluded faces can be placed in a separate category or assigned to a cluster only when confidence is sufficiently high.

2.3 Accuracy vs Computational Efficiency

There is an important trade-off between recognition accuracy and processing speed.

High-Accuracy Approach

- Uses large deep learning models
- Processes high-resolution images
- Performs detailed face comparison
- Provides better accuracy
- Requires more memory and processing time

Efficient Local Approach

- Uses lightweight models
- Resizes images before processing
- Uses quantized models
- Processes images in batches
- Stores face embeddings instead of repeatedly processing images
- Uses approximate nearest-neighbor search

A practical approach is to use a **two-stage system**:

1. Fast lightweight processing for all images

2. More accurate processing only for uncertain cases

This improves efficiency while maintaining good recognition accuracy.

---

3. Smart City Surveillance and Pedestrian Tracking

Scenario

A smart city surveillance system must detect moving objects and track pedestrians across multiple cameras.

Constraints:

- Crowded scenes with frequent occlusion
- Limited network bandwidth
- Real-time processing

3.1 Foreground–Background Separation

A robust foreground–background separation system can use **adaptive background modeling**.

Proposed Method

1. Initialize a background model.
2. Update the background model gradually over time.
3. Compare the current frame with the background model.
4. Mark significantly different regions as foreground.
5. Apply morphological operations to remove noise.
6. Use temporal consistency to confirm moving objects.

An adaptive Gaussian Mixture Model (GMM) can represent different background states, such as moving trees or changing lighting.

To minimize false positives:

- Use motion information from consecutive frames.
- Ignore very small regions.
- Apply shadow removal.
- Use morphological opening and closing.
- Combine motion detection with object detection.

Pipeline:

Video Frame → Adaptive Background Model → Foreground Mask → Noise Removal → Shadow Removal → Object Detection → Tracking

This approach reduces false positives caused by lighting changes, shadows, and background movement.

3.2 Particle Filters for Occlusion Handling

A particle filter represents the possible states of a pedestrian using multiple particles.

Each particle may contain:

- Position
- Velocity
- Direction

During Normal Tracking

Particles are concentrated around the detected pedestrian.

During Occlusion

If the pedestrian is temporarily hidden:

1. The system predicts future positions using the motion model.
2. Particles spread across possible locations.
3. The pedestrian continues to be tracked even without direct detection.
4. When the pedestrian reappears, particles near the correct position receive higher weights.

Therefore, particle filters are useful because they can represent uncertainty and multiple possible movement paths.

3.3 Efficient Multi-Camera Pedestrian Identification

Each camera should perform local processing to reduce network bandwidth.

Proposed Strategy

1. Detect pedestrians locally.
2. Extract a compact person Re-Identification (Re-ID) feature vector.
3. Track each person locally.
4. Send only:
    - Person ID
    - Timestamp
    - Camera ID
    - Compact feature embedding

5. Compare embeddings only when a person moves between nearby cameras.

A central system can match pedestrian embeddings across different viewpoints.

Additional information can include:

- Time of movement
- Walking direction
- Camera topology
- Expected travel time

This avoids sending full video streams and provides efficient multi-camera tracking.

---

4. Self-Driving Car Navigation

Scenario

A self-driving car must detect roads, lanes, road signs, and pedestrians.

Constraints:

- Rain, shadows, and nighttime conditions
- Processing time of less than or equal to 50 ms per frame
- High-density traffic

4.1 Robust Road Detection

A robust road detection system should combine multiple image features.

Preprocessing

- Gamma correction for dark images
- CLAHE for contrast enhancement
- Rain and noise removal
- Shadow detection and removal

Feature Extraction

Use:

- Color information
- Edge detection
- Lane markings
- Texture
- Perspective information

For improved reliability, sensor fusion can combine:

- RGB cameras
- Infrared cameras
- Radar
- LiDAR

At night, infrared cameras and temporal information can improve detection. In rain, temporal filtering can reduce the effect of raindrops.

Pipeline:

Camera Input → Image Enhancement → Shadow/Rain Removal → Lane and Road Feature Extraction → Temporal Tracking → Road Area Detection

---

4.2 Efficient Pipeline for Road Signs and Pedestrians

A shared deep learning backbone can process both tasks efficiently.

Proposed Pipeline

Input Frame

↓

Image Preprocessing

↓

Shared Lightweight CNN Backbone

↓

Feature Pyramid

↙ ↘

Road Sign Detection Pedestrian Detection

↓

Object Tracking

↓

Driving Decision System

Using a shared backbone reduces repeated computations.

Additional optimization techniques include:

- Image resizing
- Quantization

- Pruning
- Knowledge distillation
- Tensor acceleration
- Region-of-interest processing

Pedestrians and road signs can also be detected only in relevant image regions when possible.

4.3 Deep Learning vs Classical Computer Vision

Aspect	Deep Learning	Classical Computer Vision
Accuracy	Usually high	Moderate
Robustness	Good after proper training	Sensitive to conditions
Training Data	Requires large datasets	Less data required
Computation	High	Low
Weather Handling	Can be robust with diverse training	Often affected
Real-Time Speed	Requires optimization	Usually faster
Hardware	GPU/NPU preferred	Can run on CPU

Deep Learning Advantages

- Better object detection accuracy
- Handles complex scenes
- Detects pedestrians and signs simultaneously
- Learns useful features automatically

Classical Vision Advantages

- Lower computational cost
- Faster on simple hardware
- Easier to implement for simple tasks

A **hybrid approach** is recommended:

- Use classical vision for simple tasks such as edge and lane detection.
- Use lightweight deep learning models for pedestrian and road-sign detection.

This provides a good balance between accuracy and real-time performance.

---

5. AR System for Surgical Assistance

Scenario

An AR system must overlay segmented organs in real time using CT or MRI data.

Constraints:

- High-resolution 3D medical images
- Latency below 100 ms
- Limited GPU memory on a wearable AR headset

5.1 Real-Time Organ Segmentation

A lightweight segmentation pipeline can be used.

Proposed Method

1. Preprocess CT/MRI images.
2. Resize or divide the 3D volume into smaller patches.
3. Use a lightweight 3D or 2.5D segmentation model.
4. Perform inference using reduced precision.
5. Combine the segmented patches.
6. Generate a simplified 3D organ representation.

Useful optimization techniques include:

- Lightweight U-Net architecture
- Reduced input resolution
- Patch-based processing
- Model pruning
- Quantization
- FP16 or INT8 inference
- GPU acceleration
- Region-of-interest segmentation

A coarse-to-fine approach can also be used:

Low-Resolution Segmentation

→ Identify Region of Interest

→ High-Resolution Segmentation of Important Area

This reduces memory usage while preserving important anatomical details.

5.2 Accurate AR/MR Overlay Integration

Accurate overlays require correct registration between medical images and the patient.

The system can use:

1. **Patient-to-Image Registration**
2. **Optical Tracking**
3. **Surface or Fiducial Markers**
4. **Real-Time Headset Tracking**
5. **Depth Estimation**
6. **Continuous Registration Correction**

The process is:

CT/MRI Image → Organ Segmentation → 3D Model Creation → Coordinate Registration → Tracking → AR/MR Overlay

The overlay should continuously update as the surgeon or headset moves.

To improve accuracy:

- Use anatomical landmarks.
- Apply rigid or deformable registration.
- Use sensor fusion.
- Calibrate the AR display accurately.
- Track patient movement in real time.

5.3 Reducing Computational Load While Maintaining Accuracy

The following strategies can reduce computational requirements:

1. Model Compression

Use pruning to remove unnecessary model parameters.

2. Quantization

Convert FP32 models to FP16 or INT8.

3. Knowledge Distillation

Train a smaller model using a larger and more accurate teacher model.

4. Region-of-Interest Processing

Process only the anatomical region needed for the current procedure.

5. Adaptive Resolution

Use high resolution only near important boundaries.

6. Frame Reuse

Reuse segmentation results when there is little movement.

7. Multi-Resolution Processing

Perform global analysis at low resolution and detailed analysis at high resolution.

8. Hardware Acceleration

Use GPU, NPU, or dedicated AI accelerators when available.

Conclusion: A lightweight compressed segmentation model combined with region-of-interest processing and adaptive resolution can satisfy the memory and latency constraints while maintaining good segmentation accuracy.

Overall Conclusion

Constraint-based computer vision problem solving requires selecting algorithms according to available resources, accuracy requirements, latency, environmental conditions, and privacy constraints.

Across all five scenarios, the key strategies are:

- Effective image preprocessing
- Feature reduction and model optimization
- Lightweight deep learning models
- Classical computer vision where appropriate
- Temporal tracking
- Hardware acceleration
- Local processing for privacy
- Adaptive computation based on confidence and scene complexity

By combining these techniques, reliable computer vision systems can be developed even under strict constraints such as limited memory, real-time requirements, low-light conditions, occlusion, and high-resolution data.